

CS7CS2 – Optimisation for Machine Learning

Student Name: Stephen Komolafe

Student Number: 21336975

Week 2 Assignment

Q1.

- (a) The variable x is defined as a symbolic variable using the SymPy library.

```
import sympy as sp
x = sp.symbols('x')
f = x**4
dfdx = sp.diff(f, x)
print("df/dx = " + str(dfdx))
```

The function $f(x) = x^4$ is then specified symbolically. The derivative is computed using SymPy's `diff()` function and stored in `dfdx`. Evaluating this expression returns $4x^3$, the exact derivative of x^4 .

- (b) Functions were defined to compute $f(x) = x^4$, the exact derivative $f'(x) = 4x^3$, and the forward finite-difference approximation:

$$\frac{f(x+\delta)-f(x)}{\delta}, \quad \text{where } \delta = 0.01.$$

```
import sympy as sp
import sympy.plotting as plt

def f(x):
    return x**4

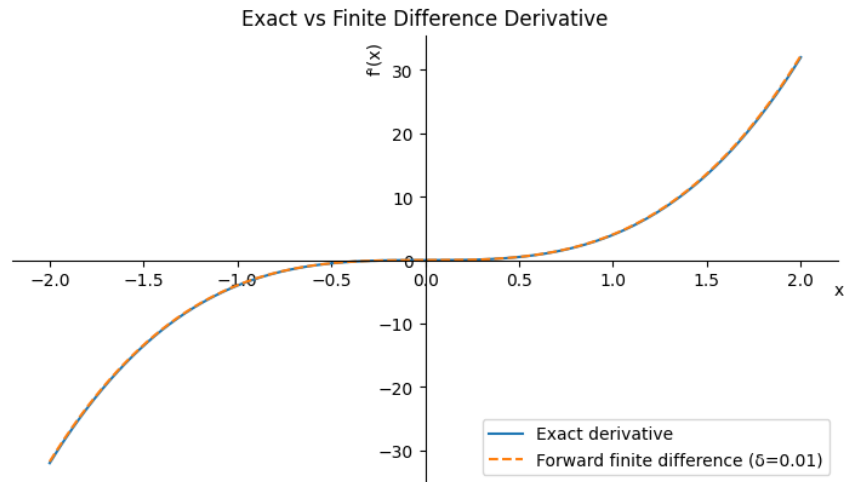
def df_exact(x):
    return 4*x**3

def df_forward(x, delta):
    return (f(x + delta) - f(x)) / delta

# range
x_vals = np.linspace(-2, 2, 400)
delta = 0.01

exact = df_exact(x_vals)
approx = df_forward(x_vals, delta)
```

The derivative was evaluated over the interval $x \in [-2, 2]$ using 400 evenly spaced points. Both the exact derivative and the forward finite-difference approximation were plotted on the same graph.



The resulting plot shows that the two curves overlap closely across the interval, indicating that the forward finite-difference method provides a good approximation to the exact derivative for $\delta = 0.01$.

- (c) To evaluate the accuracy of the forward finite-difference approximation, the Mean Absolute Error (MAE) was computed:

$$MAE = \frac{1}{N} \sum_{i=1}^N |f'_{exact}(x_i) - f'_{approx}(x_i)|$$

Where f'_{exact} is the actual derivative, f'_{approx} is the forward finite-difference approximation, and N is the number of deltas to evaluate.

The MAE was calculated for $\delta \in \{0.001, 0.01, 0.1, 1\}$.

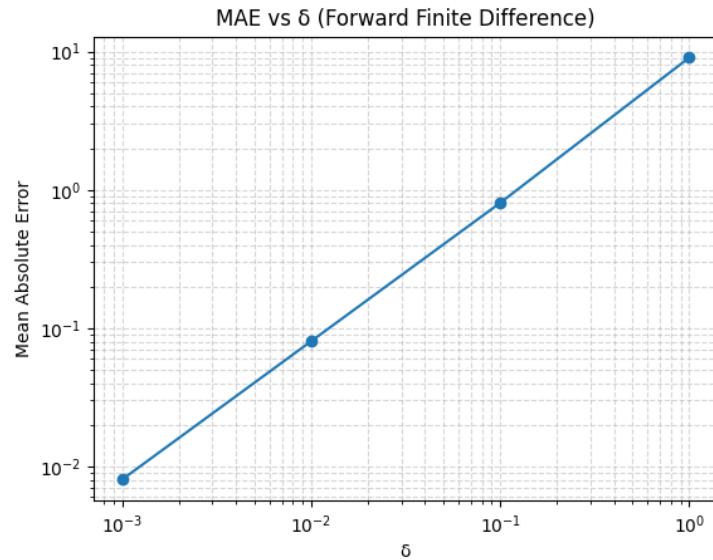
```
import numpy as np
deltas = [0.001, 0.01, 0.1, 1] # 0.001 to 1
mae = []

print("Delta\t\tMAE")
print("-" * 25)

for delta in deltas:
    approx = df_forward(x_vals, delta)
    error = np.mean(np.abs(exact - approx))
    mae.append(error)
    print(f"{delta:.3f}\t\t{error:.3f}")
```

Delta (δ)	MAE
0.001	0.008
0.010	0.080
0.100	0.815
1.000	9.040

The results show that the error increases proportionally to δ . This behaviour is expected, since larger step sizes introduce greater truncation error in the finite-difference approximation.



Plotting the MAE against δ further proves this trend, demonstrating a uniform increase in error as the step size grows.

(d) Gradient descent was implemented using the update rule:

$$x_{k+1} = x_k - \alpha f'(x_k)$$

for a total of ten iterations, with the initial value $x_0 = 1$, and learning rates $\alpha = 0.05, 0.5, 1.2$.

```
def gradient_descent(alpha, x0=1.0, n_iters=10):
    x = x0
    xs = [x]
    fs = [f(x)]

    for _ in range(n_iters):
        x = x - alpha * df_exact(x)
        if abs(x) > 1e6: #divergence occurs
            print(f"Divergence detected for alpha = {alpha}")
            break

        xs.append(x)
        fs.append(f(x))
        print(f"{x:.3f}\t{f(x):.3f}")

    print('\n')
    return np.array(xs), np.array(fs)

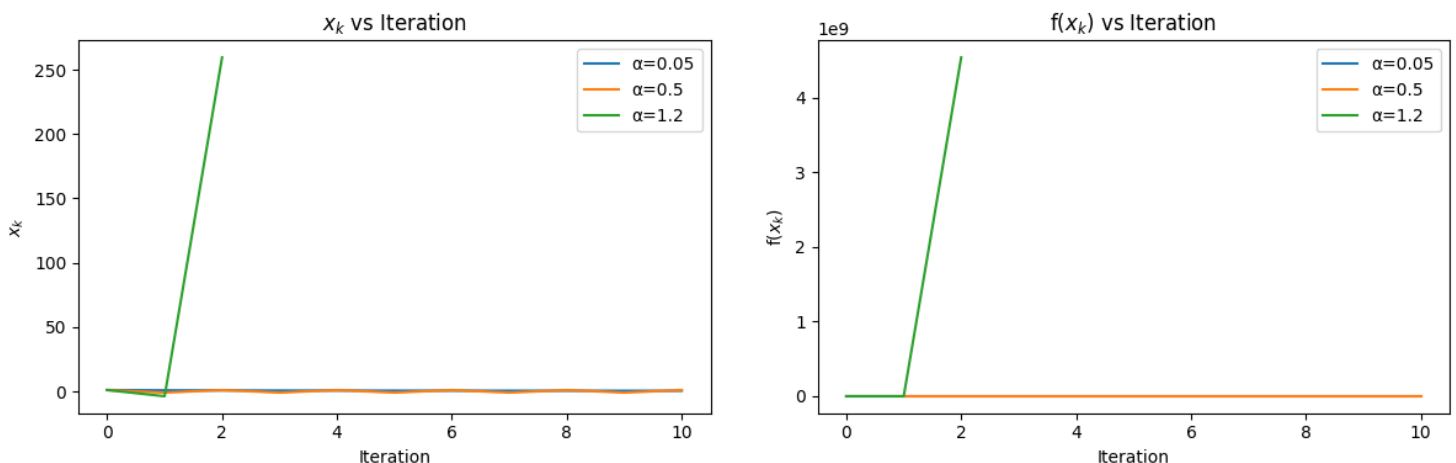
alphas = [0.05, 0.5, 1.2]
results = {}

for alpha in alphas:
    print(f"learning rate: {alpha}")
    print("x \t\t f(x)")
    print("-" * 25)
    results[alpha] = gradient_descent(alpha)
```

learning rate: 0.05		learning rate: 0.5		learning rate: 1.2	
x	f(x)	x	f(x)	x	f(x)
0.800	0.410	-1.000	1.000	-3.800	208.514
0.698	0.237	1.000	1.000	259.586	4540695601.036
0.630	0.157	-1.000	1.000	Divergence detected for alpha = 1.2	
0.580	0.113	1.000	1.000		
0.541	0.086	-1.000	1.000		
0.509	0.067	1.000	1.000		
0.483	0.054	-1.000	1.000		
0.460	0.045	1.000	1.000		
0.441	0.038	-1.000	1.000		
0.424	0.032	1.000	1.000		

At each iteration, the updated value of x_k and the corresponding function value $f(x_k)$ were stored. A divergence check was included to terminate the iteration if $|x_k|$ exceeded a large threshold, preventing numerical overflow.

(e) The values of x_k and $f(x_k)$ were plotted against iteration number for each learning rate:



For $\alpha = 0.05$, the method converges smoothly toward the global minimum at $x = 0$. Both x_k and $f(x_k)$ decrease monotonically, indicating stable but relatively slow convergence.

For $\alpha = 0.5$, the iterates oscillate between $x = -1$ and $x = 1$. The function value remains constant at $f(x_k) = 1$, showing that the method fails to converge due to an overly large step size.

For $\alpha = 1.2$, the iterates grow rapidly in magnitude, and the method diverges. This demonstrates how large learning rates can lead to instability and failure of gradient descent.

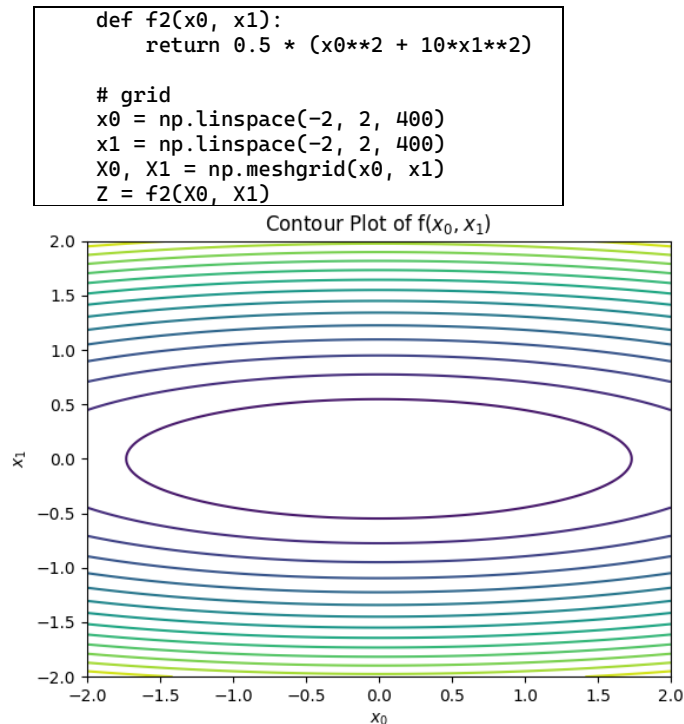
Q2.

(I)

- (a) A contour plot was generated over the domain $(x_0, x_1) \in [-2, 2] \times [-2, 2]$ by evaluating the function:

$$f(x_0, x_1) = \frac{1}{2}(x_0^2 + 10x_1^2)$$

on a grid of points using NumPy and visualising the level sets with Matplotlib's `contour()` function.



The resulting contours form elongated ellipses centred at the origin. The elongation occurs because the curvature in the x_1 direction is ten times larger than in the x_0 direction. Consequently, the function increases much more rapidly along x_1 than along x_0 . The global minimum is located at (0, 0). This anisotropic curvature creates a narrow valley structure, which directly influences the behaviour of gradient descent.

(b) Gradient descent was implemented using the update rule:

$$x_{k+1} = x_k - \alpha \nabla f(x_k), \quad \text{where} \quad \nabla f(x_0, x_1) = [x_0, 10x_1].$$

Starting from $x^{(0)} = [1.5, 1.5]$, the algorithm was run with learning rates $\alpha = 0.05$ and $\alpha = 0.2$.

```
def grad_f(x):
    return np.array([x[0], 10*x[1]])

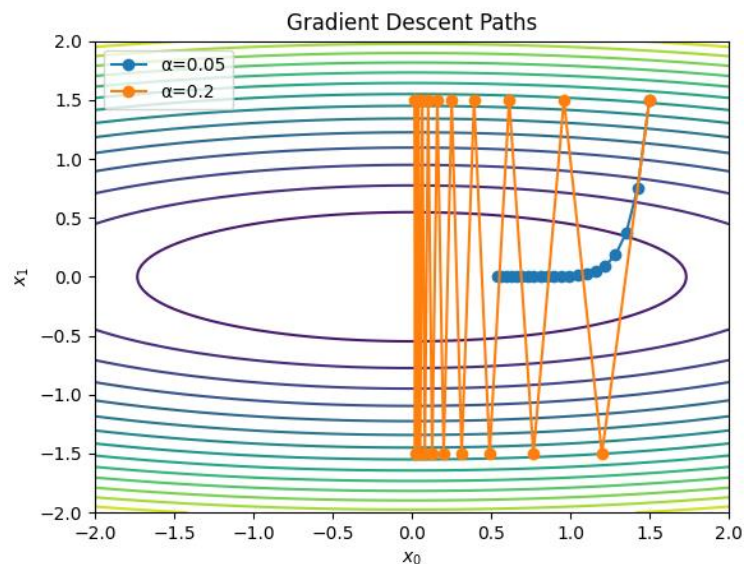
def gradient_descent_2d(alpha, x0, n_iters=20):
    x = np.array(x0, dtype=float)
    path = [x.copy()]

    for _ in range(n_iters):
        x = x - alpha * grad_f(x)
        path.append(x.copy())

    return np.array(path)

x_init = [1.5, 1.5]
path1 = gradient_descent_2d(0.05, x_init)
path2 = gradient_descent_2d(0.2, x_init)
```

The resulting paths were overlaid on the contour plot.



For $\alpha = 0.05$, the iterates converge toward the origin in a stable manner. However, the path exhibits a zig-zag pattern as it approaches the minimum. This behaviour occurs because the gradient component in the x_1 direction is much larger, causing updates to overshoot across the narrow valley before gradually stabilising.

For $\alpha = 0.2$, the oscillations across the valley become more pronounced. The larger step size amplifies the imbalance in curvature, leading to stronger zig-zag behaviour and slower practical progress toward the minimum.

This demonstrates the narrow valley effect. That being when curvature differs significantly across directions, gradient descent may oscillate across the steep direction while progressing slowly along the shallow direction.

(III)

(a) The function:

$$f(x) = x^4 - 2x^2 + 0.1x$$

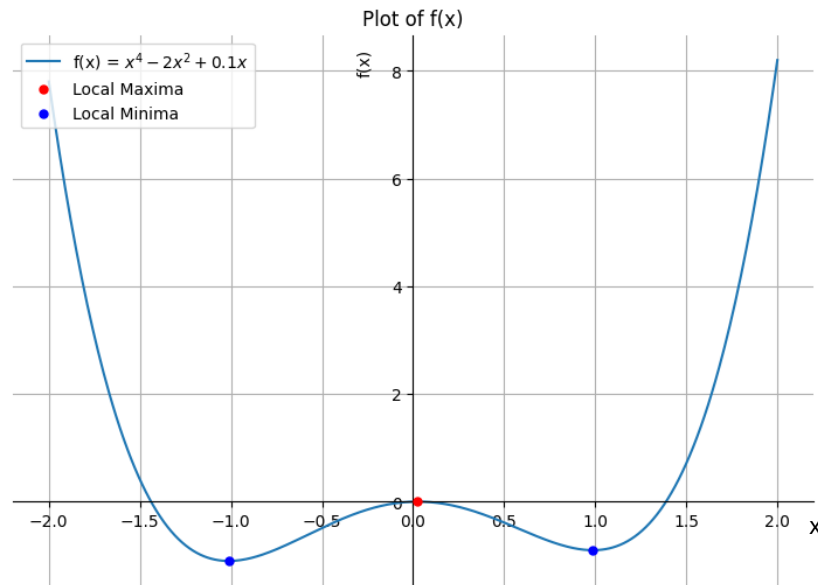
was plotted over the interval $[-2, 2]$.

```
from scipy.signal import argrelextrema

def f3(x):
    return x**4 - 2*x**2 + 0.1*x

x_vals = np.linspace(-2, 2, 400)
y_vals = f3(x_vals)
max_indices = argrelextrema(y_vals, np.greater)[0]
min_indices = argrelextrema(y_vals, np.less)[0]
```

Additionally, the local minima and maxima of the function were included to further analyse the generated plot.



The graph shows a non-convex shape with two local minima separated by a local maximum. The linear term $0.1x$ appears to introduce asymmetry, causing the two minima to have slightly different depths. This confirms that the function contains multiple stationary points and is not globally convex.

(b) The functions derivative:

$$f'(x) = 4x^3 - 4x + 0.1$$

was used to implement gradient descent with $\alpha = 0.05$. The algorithm was run from two different initial points: $x_0 = -1.5$ and $x_0 = 1.5$.

```
def df3(x):
    return 4*x**3 - 4*x + 0.1

def gradient_descent_1d(alpha, x0, n_iters=50):
    x = x0
    path = [x]

    for _ in range(n_iters):
        x = x - alpha * df3(x)
        path.append(x)

    return np.array(path)

path_neg = gradient_descent_1d(0.05, -1.5)
path_pos = gradient_descent_1d(0.05, 1.5)

print("Final from -1.5:", path_neg[-1])
print("Final from 1.5:", path_pos[-1])
```

The final solutions computed were:

@ $x_0 = -1.5$	$x \approx -1.0123$
@ $x_0 = 1.5$	$x \approx 0.9873$

These final solutions are close to two different local minima of the function. Although both points correspond to minima, they are in different regions of the domain. Since the function is non-convex, gradient descent does not guarantee

convergence to a global minimum. Instead it converges to the local minimum within the *basin of attraction* of the initial point.

Q3.

(I)

- (a) Gradient descent of the quadratic function $f(x) = x^2$ was implemented using the update rule:

$$x_{k+1} = x_k - \alpha f'(x_k)$$

$$\therefore x_{k+1} = x_k - \alpha (2x_k),$$

starting from $x_0 = 1$ for 30 iterations. The algorithm was run for step sizes $\alpha = 0.1, 0.01, 1.01$.

```
def f_quad(x):
    return x**2

def df_quad(x):
    return 2*x

def gradient_descent_1d(alpha, x0=1, n_iters=30):
    x = x0
    xs = [x]
    fs = [f_quad(x)]
    for _ in range(n_iters):
        x = x - alpha * df_quad(x)
        xs.append(x)
        fs.append(f_quad(x))

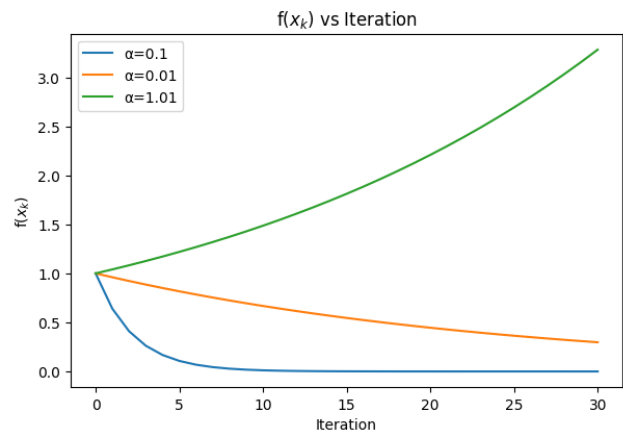
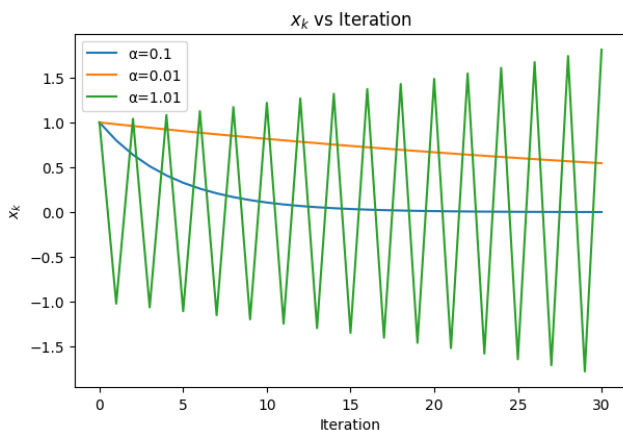
    return np.array(xs), np.array(fs)

alphas = [0.1, 0.01, 1.01]
results = {}

for alpha in alphas:
    results[alpha] = gradient_descent_1d(alpha)
```

Both the x_k and function values $f(x_k)$ are stored in arrays for plotting.

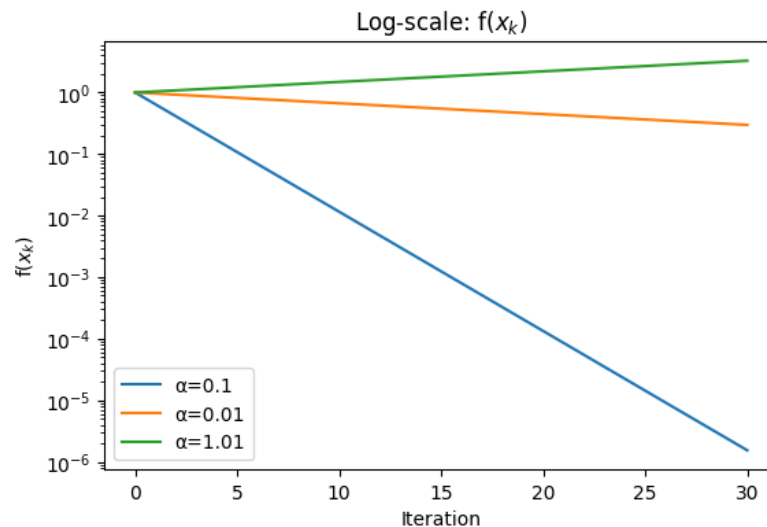
- (b) Observations remain consistent between the plots of x_k and $f(x_k)$ versus iteration.



For $\alpha = 0.1$, the iterates converge rapidly to zero, while for $\alpha = 0.01$, convergence is much slower due to the smaller step size. For $\alpha = 1.01$, the iterates diverge and grow in magnitude in both plots. This occurs because the update rule can be written as:

$$x_{k+1} = (1 - 2\alpha)x_k.$$

Convergence requires $|1 - 2\alpha| < 1$, which only holds true when $0 < \alpha < 1$. Since $\alpha = 1.01$ violates this condition, the magnitude of the multiplier exceeds one and the iterates increase exponentially.



The log-scale plot of $f(x_k)$ clearly illustrates exponential convergence for $\alpha = 0.1$ and $\alpha = 0.01$, appearing as straight lines on the logarithmic axis. Using a log-scale instead of a standard plot makes differences in convergence rates visible and clearly highlights divergence for $\alpha = 1.01$.

(II)

- (a) Gradient descent of the scaled quadratic function $f(x) = \gamma x^2$ was implemented using the update rule:

$$x_{k+1} = x_k - \alpha (2\gamma x)$$

$$\therefore x_{k+1} = (1 - 2\alpha\gamma)x_k, \quad \text{where } \gamma \in \{0.5, 1, 2, 5\}.$$

```

gammas = [0.5, 1, 2, 5]
def f_gamma(x, gamma):
    return gamma * x**2

def gradient_descent_gamma(alpha, gamma, x0=1, n_iters=30):
    x = x0
    fs = [f_gamma(x, gamma)]

    for _ in range(n_iters):
        x = x * (1 - (2*alpha*gamma))
        fs.append(f_gamma(x, gamma))

    return np.array(fs)

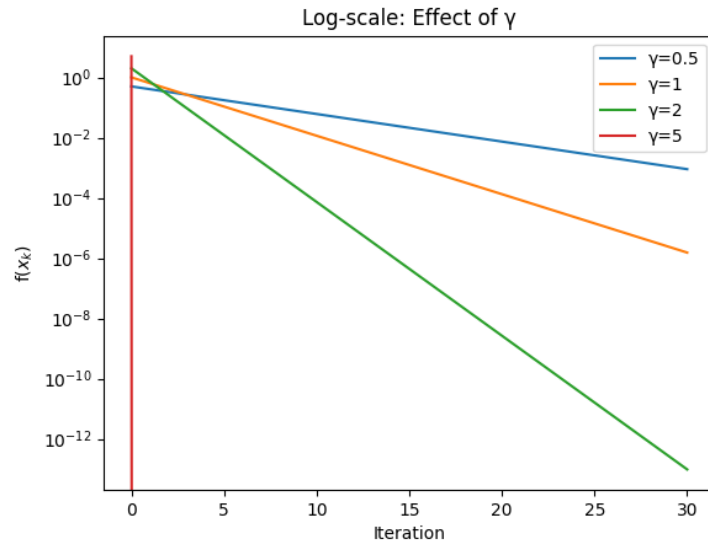
results_gamma = {}

for gamma in gammas:
    results_gamma[gamma] = gradient_descent_gamma(0.1, gamma)

```

The $f(x_k)$ values from running gradient descent on each γ are stored in an array for plotting.

- (b) The log-scale plots of $f(x_k)$ versus iteration demonstrate that increasing γ accelerates convergence.



For small γ , convergence is slower because the effective multiplier $1 - 2\alpha\gamma$ is close to one. As γ increases, the multiplier decreases in magnitude, leading to faster decay. When $\gamma = 5$, the factor becomes zero, causing convergence in a single iteration. This illustrates how curvature influences the stability and speed of gradient descent.

(III)

- (a) Gradient descent of the non-differentiable function $f(x) = |x|$ was implemented using the sub-gradient:

$$g(x) = \text{sign}(x), \quad g(0) = 0.$$

Starting from $x_0 = 1$ with $\alpha = 0.1$, the algorithm was run for 60 iterations. Since the subgradient has constant magnitude 1 (except at zero), each update changes the iterate by a fixed amount of 0.1.

```
def subgrad(x): #sign(x) func
    if x > 0:
        return 1
    elif x < 0:
        return -1
    else:
        #x=0
        return 0

def gradient_descent_abs(alpha, x0=1, n_iters=60):
    x = x0
    xs = [x]
    fs = [abs(x)]

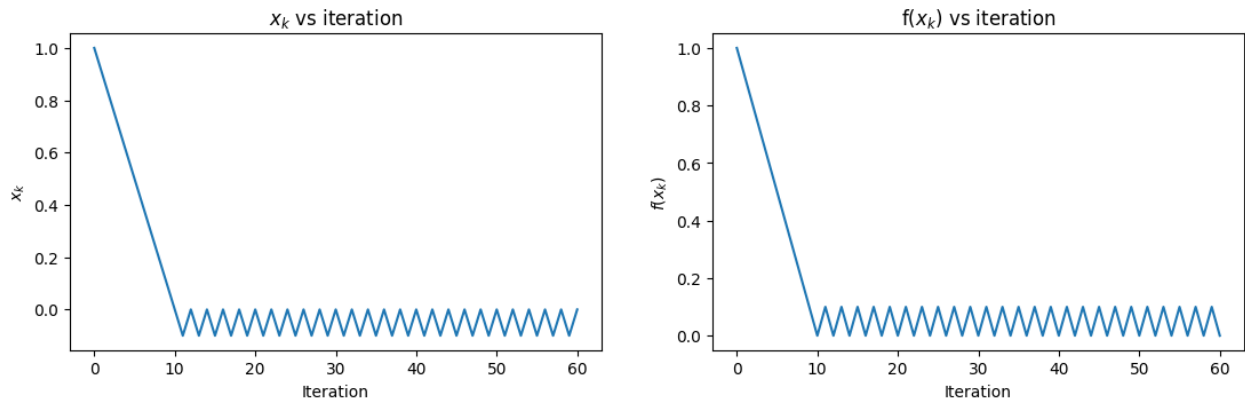
    for _ in range(n_iters):
        x = x - alpha * subgrad(x)
        xs.append(x)
        fs.append(abs(x))

    return np.array(xs), np.array(fs)

xs_abs, fs_abs = gradient_descent_abs(0.1)
```

The magnitudes of x_k and $f(x_k)$ are stored in an array for plotting.

(b) Plots of x_k and $f(x_k)$ vs iterations were generated:



The plot of x_k shows linear movement toward zero until the iterate crosses the origin. After crossing zero, the sign of the sub gradient reverses, causing the method to overshoot in the opposite direction. As a result, the iterates oscillate between positive and negative values near zero instead of converging exactly. The plot of $f(x_k)$ mirrors this oscillatory behaviour, flattening near zero but never reaching it exactly.

Q4.

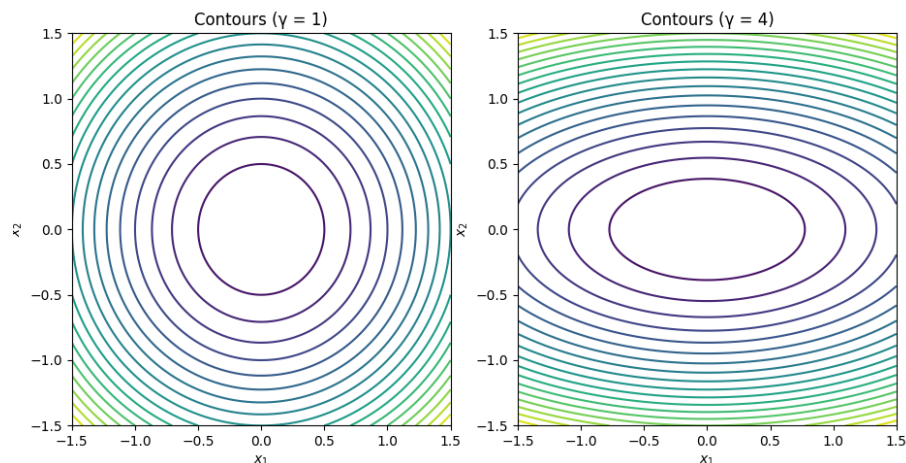
(a) A contour plot was generated over the domain $[-1.5, 1.5]^2$ for $\gamma = 1$ and $\gamma = 4$ by evaluating the function:

$$f(x_1, x_2) = x_1^2 + \gamma x_2^2$$

```
def f_q4(x1, x2, gamma):
    return x1**2 + gamma * x2**2

x1 = np.linspace(-1.5, 1.5, 400)
x2 = np.linspace(-1.5, 1.5, 400)
X1, X2 = np.meshgrid(x1, x2)
gammas = [1, 4]
plt.figure(figsize=(12,5))

for i, gamma in enumerate(gammas):
    plt.subplot(1,2,i+1)
    Z = f_q4(X1, X2, gamma)
    plt.contour(X1, X2, Z, levels=20)
    plt.title(f"Contours ( $\gamma = \{gamma\}$ )")
    plt.xlabel("$x_1$")
    plt.ylabel("$x_2$")
```



For $\gamma = 1$, the contours are circular, indicating equal curvature in both directions. For $\gamma = 4$, the contours become elongated ellipses, compressed in the x_2 direction due to the larger curvature and thus demonstrating how increasing γ creates anisotropic curvature.

(b) Gradient descent was implemented using the update:

$$x_{k+1} = x_k - \alpha \nabla f(x_k)$$

with $\alpha = 0.1$ and the starting point $x^{(0)} = [1, 1]$.

```
def grad_q4(x, gamma):
    return np.array([2*x[0], 2*gamma*x[1]])

def gradient_descent_q4(alpha, gamma, x0, n_iters=20):
    x = np.array(x0, dtype=float)
    path = [x.copy()]

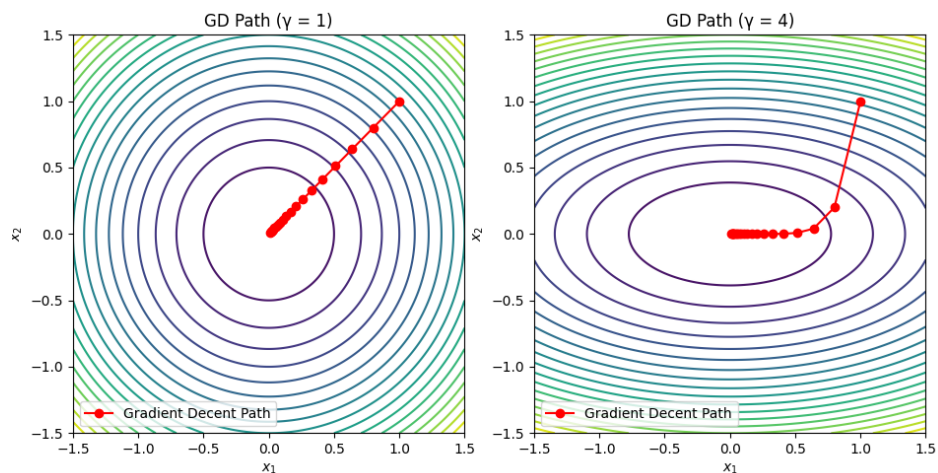
    for _ in range(n_iters):
        x = x - alpha * grad_q4(x, gamma)
        path.append(x.copy())

    return np.array(path)

x_init = [1, 1]
alpha = 0.1
paths = {}

for gamma in [1, 4]:
    paths[gamma] = gradient_descent_q4(alpha, gamma, x_init)
```

The paths were stored and overlaid on the contour plots.



For $\gamma = 1$, the iterates move smoothly and directly toward the origin, shrinking uniformly in both coordinates. For $\gamma = 4$, convergence in the x_2 direction is faster due to the larger gradient component, while progress in x_1 is slower. The path therefore bends toward the coordinate axes before reaching the origin.

(c) A contour plot was generated over the domain $[-2, 2] \times [-1, 3]$ by evaluating the Rosenbrock function:

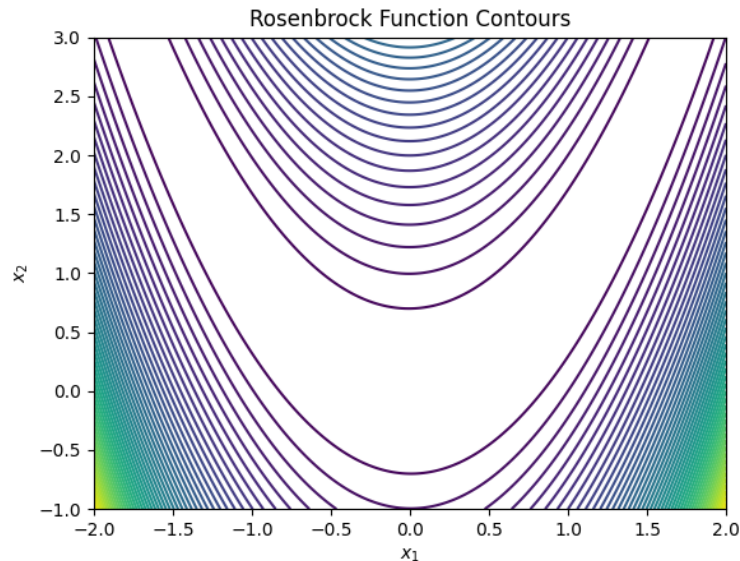
$$f(x_1, x_2) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2,$$

on a meshgrid of points using NumPy.

```
def rosenbrock(x1, x2):
    return (1 - x1)**2 + 100*(x2 - x1**2)**2

x1 = np.linspace(-2, 2, 400)
x2 = np.linspace(-1, 3, 400)
X1, X2 = np.meshgrid(x1, x2)
Z = rosenbrock(X1, X2)
```

These points could then be visualising to produce the following contour plot.



The contour plot reveals a narrow, curved valley with a global minimum at (1,1). The valley is steep across its width and shallow along its length, making the function highly ill-conditioned. This structure is known to cause difficulties for basic gradient descent methods.

- (d) Gradient descent was applied from $x^{(0)} = [-1.25, 0.5]$ for 2000 iterations using step sizes $\alpha = 0.001$ and $\alpha = 0.005$.

```
def grad_rosenbrock(x):
    x1, x2 = x
    df_dx1 = -2*(1 - x1) - 400*x1*(x2 - x1**2)
    df_dx2 = 200*(x2 - x1**2)
    return np.array([df_dx1, df_dx2])

def gradient_descent_rosen(alpha, x0, n_iters=2000):
    x = np.array(x0, dtype=float)
    path = [x.copy()]

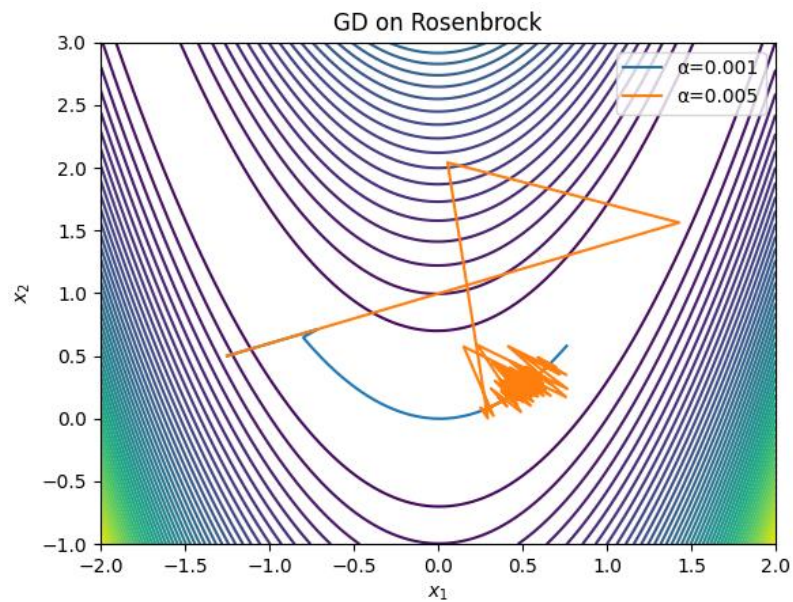
    for _ in range(n_iters):
        x = x - alpha * grad_rosenbrock(x)
        path.append(x.copy())

    return np.array(path)

x_init = [-1.25, 0.5]

path1 = gradient_descent_rosen(0.001, x_init)
path2 = gradient_descent_rosen(0.005, x_init)
```

An array of Rosenbrock gradients was computed and used to derive the optimisation paths for each step size. These paths were then plotted over the contour map.



For $\alpha = 0.001$, the algorithm converges slowly, gradually entering and following the curved valley toward the minimum at $(1, 1)$. The small step size ensures stability but results in slow progress.

For $\alpha = 0.005$, the iterates exhibit larger oscillations across the narrow valley and struggles to settle smoothly. The increased step size amplifies instability due to the steep curvature across the valley, leading to its zig-zagging behaviour.